00_机组

- 第一章：计算机基础：
  - 固件是软件，是存储在某个硬件中不在变动的软件
  - 三个不知道哪里来的定义
    - MTBF：平均第一次工作时间，值越大越好，是从第一次使用开始到出故障的平均时间
    - MTTF：平均失效之前工作的时间，值越大越好，代表两次失效之间实际工作的时间
    - MTTR：平均修复时间，值越小越好
  - 主频并非越大越好，越大的主频对应越高的功耗
  - 冯诺依曼的基本特点：按地址访问并顺序执行指令
  - 指令时间计算：
    - CPI*（1/Hz）=一条指令的实际消耗时间
    - IPS=1/（CPI*（1/Hz））=(1/Hz)/CPI=主频/CPI
- 第二章：存储器
  - 芯片线路计算：
    - 地址线：如果DRAM采用地址线复用，则除2计算
    - 数据线
    - CE片选线：如果采用DRAM复用，则是两条（行选通，列选通），否则是1条
    - 读写控制线：1条或2条
  - 关于存储器的总体构成框架如下：
    - 内存
      - 寄存器
      - cache
      - 主存：（主存包括ROM和RAM，bios算作主存）
        - 接口卡中的ROM
        - 显卡中的RAM

            - 内存条
    - 外存：光盘，磁盘
- 每个磁道存放的信息量是一定的，因此靠内的磁道位密度较高，也因此磁盘容量受到内圈位密度制约
- 两次映射大题：
    - cache和主存的地址映射是硬件自动完成的，逻辑地址到物理地址的映射是OS完成的
    - 全相连映射一般使用相连存储器实现，相连存储器就是cache，同时制作了快表
    - 直接相连无须考虑替换问题，只有全相连和组相连会出现替换
- cache选择算法中，只有LRU是栈算法
- 全写法和回写法：
    - 读操作并不会造成数据不一致，只有写操作会，因此要通过两种方法对写进行约束
    - 全写是在cache命中时同时更改cache和主存，回写是只更新cache和脏位
    - 两者如果未命中，全写法会写分配或非写分配，回写法只会写分配，写分配的意思是分配一个cache块和主存对应

第三章：总线：

- **总线的分类
    - 片内总线（AMBA）
    - 系统总线
        - 包括数据线，地址线，控制线
    - 通信总线：
        - 同步异步是针对通信总线设定的
        - 互锁信息，同步信息都通过通信总线传输
    - IO总线：比如USB，PCI
- **总线结构
    - 单总线结构：
        - 只有系统总线，即只有数据地址控制线
        - IO和主存的统一编址使用的就是单总线结构
    - 双总线结构：

        - 主存总线
        - IO总线
    - 三总线结构：
        - 主存总线
        - IO总线
        - DMA总线：本质是主存和IO设备的总线，是另类的IO总线
    - 不同速率的总线使用桥接器进行链接
- **总线仲裁种类：**
    - 集中仲裁
        - 链式查询
            - 优先级固定，离CPU越近优先级越高，仅需一根控制线，扩展容易
            - 对硬件电路敏感
        - 计数器定时查询
            - 优先级可以改变或者不变
            - 对电路故障不敏感
        - 独立请求：响应快，次序控制灵活
    - 分布仲裁：
        - 无须中央仲裁器，有总线请求时，从自己的仲裁器发送仲裁号
        - 在自举仲裁方案下，N根请求数据线可以连接N+1个设备，具有最低优先级的设备直接接在请求线最后
            - 仲裁号大的回留在线上，最后选择当前最大仲裁号的设备请求成功
    - 仲裁结束后，获得总线控制权的设备发送总线忙信号
- **通信方式：**
    - 同步通信
    - 异步通信：三个握手
        - 异步通信时握手信息不止一位
    - 半同步通信：同时采用时钟信号和握手信号

第四章：IO

- **IO接口**
    - IO接口分类：

        - 程序查询接口
        - 中断接口
        - DMA接口
    - IO接口的寄存器和线材：
        - 数据线：（双向）
            - 链接状态控制寄存器和数据寄存器
            - 状态寄存器向CPU发送IO状态信息，控制寄存器接受CPU发送给IO的信息，两者时间上串行，共用一个寄存器
            - 数据显示传送命令状态，中断类型号，数据
        - 地址线：（一般是单向，DMA双向）
        - 控制线：（提供主存周期情况）
    - IO接口和IO端口：
        - IO接口：IO寄存器+控制逻辑（控制线提供）
            - 可以连接多个外设
            - 对IO系统的系统调用（比如读写）是由IO接口中的程序完成的
        - IO端口：IO的寄存器
            - IO编址针对的是IO端口，需要为每个端口标注端口号，给出方法对其进行访问
    - IO指令编址方式：
        - 统一编址：
            - 统一编址用于单总线结构，应用于RISC设计
            - 统一编址占用了主存地址，减少了主存地址数
            - 可以使用访存指令访问，可以使用额外方法访问
            - 执行速度慢
        - 独立编址：
            - 控制信号多，控制复杂
            - 使用输入输出指令访问
- **中断**
    - 中断分类
        - 异常（内中断）（异常无法被屏蔽）：

        - 内中断是指CPU内部产生的异常
        - 指令没有执行成功，断点是当前指令地址
        - 异常是由CPU检测时发现的，没有办法告知CPU，异常只能使用软件识别
        - 分类：
            - 硬故障中断（由硬件引发，主要是连接线，比如存储器校验错，总线错）
                - 终止：控制器出错，存储器校验错这些使得计算机无法进行
            - 软件中断：
                - 故障：（基本是统称），比如缺页缺段，操作码非法，除数0
                - 陷入（trap）：人为设计软件断点，访管中断属于这个
    - 中断（外中断，由硬件引发）：
        - 外中断是外部设备向CPU发送的中断请求
        - 指令执行成功，断点是下一条指令地址
        - 中断是由中断请求线输送告知CPU的，中断可以采用硬件识别或者软件识别
            - 硬件识别：中断向量寻址
            - 软件识别：特定软件对异常状态寄存器轮询
        - 分类：
            - 可屏蔽中断（判优优先级最低）：
            - 不可屏蔽中断（判优优先级最高）：
- 多重中断和单重中断流程：（多重中断需要保存屏蔽字）
    - 单重中断：关中断，保存断点，中断服务程序寻址/送中断向量，保存现场，中断服务程序，恢复现场，开中断
    - 多重中断：关中断，保存断点，中断服务程序寻址/送中断向量，保存现场和屏蔽字，开中断，中断服务程序，关中断，恢复现场和屏蔽字，开中断
- 中断流程解释：
    - 中断请求
    - 中断判优（高到低顺序如下）
        - 不可屏蔽中断（外中断）：机器掉电，RAM检测失败

        - 硬件异常（内中断）：一般由总线产生
        - 软件中断（内中断）：
            - 陷入：编程的断点，访管指令
            - 故障：缺页，操作数非法（一堆中断的统称）
        - 可屏蔽中断：
            - 普遍外中断：定时器中断，外部信号中断
            - DMA完成的中断
            - IO中断
            - 重启中断
    - 中断响应
        - 硬件自动完成（即使用中断隐指令完成，隐指令不是指令）
            - 关中断
            - 保存断点：保存PC和PSW，通常保存在栈，有些保存在特定寄存器
            - 中断服务程序寻址：使用硬件或软件的方法识别软件源
        - 中断程序完成（操作系统的工作）
            - 保存现场和屏蔽字：现场是用户工作寄存器的内容，一般是通用寄存器
            - 开中断
            - 执行中断服务程序
            - 关中断
            - 恢复现场和屏蔽字
            - 开中断
- **DMA**
    - DMA是一种特殊的硬件，他属于北桥芯片，南桥收集IO设备，发送给北桥处理
    - 响应DMA请求的条件时当前的机器周期执行完
    - DMA访问的三种方法：
        - CPU停止：传送一组数据块
        - 周期挪用：逐字传送
        - 交替访存：

- DMA存在两种请求：
  - DMA请求：外设发送给DMA，请求DMA控制器服务
  - 总线请求：DMA发送给CPU的请求，这并非中断请求
- ~~注意：408考纲中不包含通道方式，通道是DMA方式的升级，这里略去不表~~

第五章：数据计算

- **原反补移**：
  - 原码与反码在表示0的时候有两种表示方式，补码和移码只有一种
  - 补码在表示的时候可以多表示一个值，也因此在使用补码小数的时候，可以表示-1，而原码和反码无法表示-1
  - 主存地址使用户无符号数保存，主存并不存在负数
  - 机器零：机器零时浮点数特有的，在表示的时候下溢范围内的值和0本身都被记为机器零，即机器零是机器的零邻域部分
- **浮点计算**：
  - 数字：8,11,15，对应长度为32,64,80
- **标志位**：
  - OF：有符号数，最高位和次高位进位信息
  - CF：无符号数，最高位进位和符号位
  - SF：有符号数有效

第六章：指令

- **计算机指令格式**：
  - 如图所示

OP	X1	A1	X2	A2	X3	A3

  - 指令组成：操作码+地址码。地址码=寻址方式码X+形式地址A
  - X表示地址的寻址方式，指令的寻址方式不需要在代码中表示，如果指令要跳转执行，直接采用jmp跳转即可
- **地址寻址方式**：
  - 直接寻址：形式地址本身就是真实地址
    - 访存两次，取一次，找一次
    - 优点：简单，指令执行时只用一次访存

    - 缺点：寻址范围小，操作数地址不易修改
- 间接寻址：形式地址指向主存中另一个指针，两步找到最终地址（1次间址）

  可以扩展为多次间址

    - 访存3次，取1次，后续2次
    - 优点：可以扩大寻址范围，便于编程
    - 缺点：执行时会多次访存，效率低
- 寄存器寻址：形式地址是寄存器号，操作数在寄存器
    - 访存1次
    - 优点：可以缩短指令字，不访问主存，速度快，支持向量矩阵运算
    - 缺点：寄存器个数有限
- 寄存器间接寻址：寄存器套间接寻址
    - 2次访存：取一次，间址一次
    - 速度高于正常间接寻址
    - 寄存器间接寻址不能多次进行间接，那没有意义
- 隐含寻址：操作数隐含在ACC中
    - 优点：有利于缩短指令字长
    - 确定：需要增加存储操作数或隐含地址的硬件
- 立即寻址：指令内就是操作数
    - 这个操作数称为立即数，一般使用补码，寻址特征为#
    - 访存1次
    - 优点：速度最快
    - 缺点：指令位数会限制数据范围
- 基址寻址：形式地址加上基址寄存器（是无符号数）

  就是OS的重定位寄存器

  应用于多道程序

    - 有时会使用某个通用寄存器充当基址寄存器
    - 基址寄存器指向程序开始的地址
    - 基址寄存器面向操作系统，由操作系统或管理程序确定（无法人工修改，人工只能指定作为基址寄存器的通用寄存器）

        - 优点：便于程序浮动，方便并发运行，扩大寻址范围
    - 变址寻址：形式地址加上变址寄存器（是无符号数）

      应用于循环程序

        - 类似基址寻址，变址寄存器可以是专业的，也可以是通用寄存器
        - 变址寄存器面向用户，内容可以由用户改变
        - 相反的，我们将形式地址理解为基址，变址寄存器作为偏移量
        - 优点：便于编写循环（将i存放在IX寄存器中）
    - 相对寻址：形式地址加上PC（是有符号数）

      应用于跳转指令

        - 形式地址是相对于PC的偏移量，用补码表示
        - 优点：代码在程序内浮动时不用更改跳转指令的地址码，便于程序浮动，广泛应用于转移指令
    - 堆栈寻址：操作数存放在堆栈中
        - 使用SP指针指向栈顶元素，进行入栈出栈
        - 硬堆栈：使用寄存器实现堆栈，访存0次
        - 软堆栈：使用主存实现堆栈，访存1次
- CISC与RISC：
    - CISC复杂指令集：

      COMPLEX

        - 一条指令完成一个复杂的基本功能
        - 用于x86架构的笔记本与台式机
        - 支持更多的寻址方式
        - 采取存储程序设计，将常用的复杂指令组合方式存放起来
        - 使用微程序控制器
        - CISC指令长度长度长短不一，不适合流水线架构，但是可以实现
    - RISC精简指令集：

      REDUCED

        - 一条指令完成一个简单功能，复杂功能使用指令组合
        - 用于ARM架构的手机与平板

        - 通用寄存器的数量更多
        - RISC指令长度格式长短大致相同，且必须实现流水线架构
        - 内部寄存器数量多，使用硬布线
        - 使用复合逻辑控制器
        - RISC兼容性差，因为在此之前intel使用CISC已经市场垄断了

- # 第七章：CPU
    - **数据通路**
        - 组合逻辑元件：选择器，加法器，ALU，译码器
        - 存储元件（时序逻辑电路）：暂存寄存器，通用寄存器（时序逻辑是因为三态门和暂存寄存器有输入端口，故被称为时序逻辑）
    - **程序员可见寄存器**：
        - 可见寄存器：PC，通用寄存器，PSW，虚拟存储器，变址寄存器
        - 不可见寄存器：MAR，MDR，IR，cache，暂存寄存器DR
        - 基址寄存器：可以修改基址寄存器由哪个寄存器担任，但是无法修改值
        - 对于其中的PC和PSW而言，可见并不意味着可以获取两者存入的值，只能对已有位置的值进行写入，比如mov指令改变PC的值，但是没有指令可以获取当前PC的值
    - **冒险**：
        - 结构冒险（硬件同时访问，多指令请求临界资源）
            - 解决办法：多端口寄存器，分离型cache（哈佛结构存储器），预取指令，L/S指令风格，多体交叉存储器
        - 数据冒险：
            - 写后读（只有这个发生在标准流水线）
            - 读后写
            - 写后写
            - 解决方法：数据旁路，插入空指令，插入空泡，编译时调整指令顺序
        - 控制冒险：只发生在jmp指令
            - 解决方法：分支预测，延迟转移，提前形成条件码，插入空指令/空泡
    - **额外**
        - 水平竖直微指令：

- 水平：微程序短，微指令长，整体执行速度快，编码空间利用率低，并行度高
- 竖直：微程序长，微指令短，执行速度慢，效率差，并行度低